ISC2 SECURITY CONGRESS 2026

ISC2.org/Congress | #ISC2Congress

From Zero to Security Bot

Building and Hardening an AI-Powered Code Review Pipeline

Pre-Conference Workshop, Part 2: Sunday, October 25, 2026

Cameron Hopkin

Founder, Waypoint Compliance Advisory



Capstone: Watch the Bot Think

The Pull Request

A new invoice endpoint. Would you approve it?

```
@app.route("/api/orders/<int:order_id>/invoice")
@auth.require_order_access
def order_invoice(order_id):
    """Return the invoice for one of the caller's orders."""
    order = models.get_order(order_id)
    if order is None:
        return jsonify({"error": "not found"}), 404
    return jsonify({"invoice": {"order_id": order["id"], "item": order["item"],
                                "total_cents": order["total_cents"]}})
```

The name says order access. The docstring says the caller's orders. Both are promises.

The Decorator It Trusts

auth.py, the file the pull request does not touch:

```
def require_order_access(view):
    """Guard an order route."""
    @wraps(view)
    def wrapper(*args, **kwargs):
        user = current_user()
        if user is None:
            return jsonify({"error": "authentication required"}), 401
        return view(*args, **kwargs)     # never checks who owns the order
    return wrapper
```

Authentication, not authorization. Any logged-in user can read any invoice: broken object-level authorization (BOLA).

Your Turn: Two Minutes

Would you approve it?

If you only saw the diff, honestly?

What would you read?

Which file, which function, before you were sure?

What will the bot do?

It sees the diff and the whole of app.py. Not auth.py.

Step 1. Diff Only

What the Model Is Sent

The sweep prompt, plus this user message (119 lines, abridged):

```
Already-known findings, do not repeat these:
none

Numbered source of target-app/app.py:
1: """Order management API for the workshop labs.
...
95: @auth.require_order_access
98:     order = models.get_order(order_id)
...
This is a pull request. It changed lines 94-109 of target-app/app.py. Review the changed
code only. The rest of the file is context. Report defects in the changed lines, or in
code they depend on.
```

"Or in code they depend on." But it cannot see that code. The decorator is just a name.

What It Answered

The raw answer, from the audit log:

```
"severity": "high",
"title": "SQL Injection via order_id parameter",
"line_start": 98,
"evidence": "order = models.get_order(order_id)",
"explanation": "... If the underlying implementation of models.get_order
   constructs a SQL query using string formatting ... this creates a SQL
   injection vulnerability ...",
"confidence": "high"
```

It missed the real bug, and invented one: a guess, rated high, with high confidence.

Why It Is Wrong, and Why It Was Kept

Wrong

<int:order_id> makes Flask turn the value into an integer.

get_order uses a parameterized query (WHERE id = ?).

The prompt says: do not guess.

Kept anyway

The citation check passed: line 98 really says order = models.get_order(order_id).

Code checks the line exists. It cannot check that the claim is true.

Lesson 1

A citation proves the line exists, not that the claim is true.

On Oct 17 the same setup failed differently: it trusted the decorator's name. Same blind spot, different symptom.

Step 2. Diff Plus Repo-Read

Stage A: The Menu

Before reviewing, the model is shown every function it may read:

```
You are about to review target-app/app.py. Readable code in this repository:
target-app/app.py: login, logout, list_orders, get_order, ..., order_invoice
target-app/auth.py: current_user, login_required, require_order_access, verify_password
target-app/db.py: get_conn, init_db, reset_db
target-app/models.py: get_user_by_username, get_order, list_orders_for_user, ...
target-app/storage.py: list_uploads, read_upload, generate_report

The pull request changed lines 94-109. Ask for the code those lines depend on.
```

Up to four reads. Names only, from this list. Test files are never on it.

Stage A: What It Asked For, and Why

```
{"file": "target-app/auth.py", "symbol": "require_order_access",
 "reason": "I need to review this decorator to confirm it correctly enforces
   authorization, ensuring the user can only access invoices for orders they own."},

{"file": "target-app/models.py", "symbol": "get_order",
 "reason": "... confirm it retrieves the correct order data and does not ... allow
   access to orders outside the intended scope."}
```

The right two functions, for the right reason: the doubt a good human reviewer has.

The Tool Runs in Code, Not in the Model

The model asked. read_symbol() in review.py decided, and did the reading.

Read-only

Named functions and classes, never a whole file.

Scoped

Only the reviewed file's folder. Python only. No test files.

Bounded

Four reads, 80 lines each.

Audited

Every request and every read is in the audit log.

Stage B: The Finding

Now the review, with both functions in front of it:

```
"title": "Broken Access Control: User can access any order by ID",
"line_start": 94, "line_end": 107,
"explanation": "... the decorator (lines 33-45 of auth.py) only checks if a user is
   logged in; it does not verify that the logged-in user owns the order ... The
   underlying models.get_order function ... explicitly returns the order regardless
   of ownership. Consequently, an authenticated user can change the URL parameter to
   any order ID and retrieve its invoice details ..."
```

The planted bug, for the right reason. And no invented SQL injection: seeing the query removed the guess.

Checked Before It Is Kept

Evidence

The quoted code is really on lines 94 to 107 of app.py.

Diff focus

The finding touches lines the pull request changed.

Allowed files

It may cite app.py, or a file the tool actually read.

Step 3. Watch It Think

Its Working Notes

Ollama returns a separate thinking field before the answer. Excerpts:

```
"The critical issue here is that models.get_order returns the order regardless of
 who owns it. The decorator @auth.require_order_access only checks for
 authentication (valid session), but doesn't check authorization (whether the user
 owns this specific order)."

"Wait, let me re-read the requirements. I need to focus on lines 94-109 ..."

"Actually, I should think about whether this is really a defect in the changed code
 or if it's just how the application works."

"So yes, this is a valid finding - an authorization bypass vulnerability ..."
```

It names the distinction, then checks itself twice before it commits.

What the Thinking Shows, and Costs

It quoted a docstring

"Returns the order regardless of who owns it." Author text: sanitizing removes it.

It slipped

It called the decorator "line 45". It meant line 95 of app.py.

It cost time

24 seconds and about 840 words, versus about 10 seconds with thinking off.

So in CI

think = false by default. Turn it on to debug a verdict, not for every pull request.

What We Measured

The capstone scorer counts a catch only if the title names access control on the endpoint or the decorator.

3 runs each	Diff only	Diff + repo-read
qwen3.5:9b, Oct 17, sanitizing off	0 of 3	**3 of 3**
qwen3.5:9b, Oct 24, after the room's rule 8	0 of 3	**3 of 3**
gpt-oss:20b, Oct 17	0 of 3	**3 of 3**
qwen3.5:9b, Oct 25, sanitizing on (docstrings stripped)	0 of 3	**3 of 3**

Lab: Do It Yourself

From the repository root, after git pull (bash shown; Windows: use the cheat sheet):

```
python eval-harness/capstone.py

# make the pull request, review it with repo-read, and see what it read
python -c "import sys, pathlib; sys.path[:0]=['eval-harness','attack-lab']; import capstone;
pathlib.Path('pr.diff').write_text(capstone.apply_pull_request(pathlib.Path('.'))[0])"
REVIEW_AUDIT_LOG=audit-capstone.jsonl python reference-pipeline/review.py --diff pr.diff --no-scanner --repo-read
python eval-harness/show_reads.py audit-capstone.jsonl

# try to break the tool's limits
python eval-harness/try_read_limits.py

git checkout target-app/app.py
```

Windows: lab-guide/WINDOWS-CHEATSHEET.md has every line, Module 8. Codespaces: capstone.py --reps 1.

Discussion

The guess

Diff-only invented a SQL injection and the citation check kept it. What check would have caught it?

The docstring

The model leaned on a true docstring. With sanitizing on it still found the bug. Does stripping comments help or hurt?

The tool

The model now chooses what to read. In Module 5, an attacker's comment steered those reads. What limit matters most?

Monday

Would you run repo-read on your team's pull requests? What would you log, and who would read the log?

Thank You!

Cameron Hopkin | Cameron@waypointca.com

github.com/cameronhopkin/isc2-2026-ai-code-review-workshop

